Class 22: Requirements Engineering for ML Systems

Session 22 of 36 – Theory and Practice: What to Build and Why

Data-driven Systems Engineering

Agenda

- Introduction, user/system requirements, and quality constraints
- Functional, non-functional, and domain requirements
- Specification styles and the requirements document
- Traceability, change, and requirement management

Note

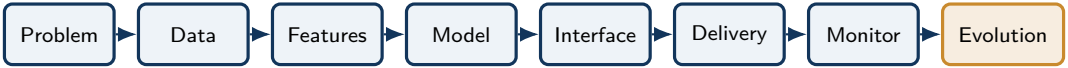
The hands-on practice follows in the second half of this class.

Why requirements are harder for ML systems

Learning goals

- Distinguish user, system, functional, and non-functional requirements.
- Write testable requirements instead of vague aspirations.
- Translate model behavior into system-level acceptance criteria.
- Connect requirements to later testing, deployment, and monitoring work.

Course Map and Running Cases



Today in the course

Focus: Requirements that make later validation possible.
Bridge: This class connects stakeholder need to technical behavior so that testing, deployment, and monitoring have something concrete to protect.

Fraud case

In the fraud case, requirements include not only prediction quality but latency, logging, explainability, and acceptable false-positive burden.

Pasteurization case

In the pasteurization case, requirements include state visibility, timely alerts, usable thresholds, and operator trust in the dashboard.

Course thread: The course thread here is that weak requirements create confusion that no later engineering step can fully repair.

Why requirements engineering matters so much

- Requirements define what success means before implementation begins.
- Weak requirements create confusion that no amount of modeling skill can repair later.
- In ML systems, requirements must cover the software, the data, the model behavior, and the operating context.

Types of requirement

User requirements

- Natural language
- Service-level descriptions
- Written for customers and operators

System requirements

- Structured and testable
- Detailed behavior and constraints
- Used by developers and reviewers

Why ML systems make requirements harder

- Behavior is partly learned from data rather than fully hand-coded.
- Labels may be delayed, noisy, or disputed.
- Model performance depends on future data conditions, not only current test data.
- Stakeholders may ask for “accuracy” when they really mean trust, fairness, speed, or interpretability.

Bad requirement vs good requirement

Weak

The fraud system should be fast, accurate, and easy to use.

Better

For a batch of 100 review requests, the API shall return a response within 800 ms for at least 95% of requests, and the review dashboard shall display the top 3 risk indicators used in scoring.

Functional, non-functional, and domain requirements

- Functional: what services the system provides.
- Non-functional: timing, reliability, security, usability, maintainability.
- Domain: constraints imposed by the operating environment.

Example

In pasteurization monitoring, "detect abnormal HeatUp behavior" is functional; "alert within 3 seconds" is non-functional; "respect sensor calibration ranges" is domain-specific.

Qualities of a good requirement

- Necessary: it supports a real stakeholder need.
- Testable: someone can verify whether it is satisfied.
- Unambiguous: it is unlikely to be interpreted in different ways.
- Feasible: it fits known technical and organizational constraints.
- Traceable: it can be linked to design, tests, and later changes.

Examples tailored to this repository

- Functional: the Flask service shall expose ‘POST /predict‘ and return JSON.
- Functional: the pasteurization dashboard shall refresh live sensor plots from the stream endpoint.
- Non-functional: model inference shall remain deterministic for a fixed artifact and input.
- Non-functional: drift alerts shall be logged with timestamp and monitored variable.

User stories are not enough by themselves

- User stories are helpful for conversation, prioritization, and scope.
- They still need acceptance criteria to become testable engineering targets.
- A story without measurable acceptance can hide major ambiguity.

Example

"As an analyst, I want a fraud score so that I can review risky transactions" is useful, but it still leaves latency, explanation, threshold logic, and failure behavior unspecified.

The software requirements document

- States what the system should do, not how it should be implemented.
- Should support communication, traceability, validation, and change control.
- Varies with context: student prototypes need less detail than regulated industrial systems.

Traceability links requirements to later work

- A requirement should connect to design choices, tests, deployment gates, and monitoring rules.
- Traceability makes change management easier because teams can see what else is affected when one requirement changes.
- It is especially valuable when the model, pipeline, and interface evolve at different speeds.

Ways to write specifications

- Natural language
- Structured natural language
- Form-based requirement templates
- Graphical models such as use cases and sequence diagrams
- Lightweight formal or tabular specifications when risk is high

Requirements management continues after release

- Requirements are not frozen forever because systems, users, and regulations change.
- Management includes versioning, prioritization, traceability, and impact analysis.
- In ML systems, drift and changing labels can indirectly trigger new requirements.

Why requirements change

- Business priorities shift.
- New data sources or interfaces appear.
- Regulations or privacy constraints tighten.
- Real users expose missing assumptions.
- The model itself changes what stakeholders ask for.

Requirements connect this course together

- Modeling needs evaluation criteria.
- Delivery pipelines need release checks and acceptance gates.
- Monitoring needs rules about what deviations matter and who should be informed.
- MLOps later depends on requirements that were explicit enough to automate.

Discussion prompt

Question

"The model should be accurate" is a common ask from stakeholders. Is it a requirement?

- This forces the distinction between an aspiration and a testable statement.
- It sets up the second half of this class, where teams turn this kind of ask into something they could actually verify.

Papers and materials

Papers and materials

IEEE recommended practice for software requirements specifications; Sommerville, Software Engineering; Practical MLOps, requirements-to-production perspective; Practical overview of functional and non-functional requirements; User stories and acceptance criteria guide.

From This Class to the Next

What stays with us

- Requirements make modeling, delivery, and monitoring choices accountable to stakeholder goals.
- Functional and non-functional expectations are both essential in ML systems.
- Traceable requirements help teams decide what to build, what to measure, and what to automate.

Project use

A project proposal becomes stronger when it can connect dataset choices, model metrics, interface behavior, and operational checks back to concrete requirements.

Next connection

The workshop that follows puts this theory into a written, testable requirements artifact.

Course thread: Explicit expectations make it possible to choose a realistic development process, which is where the course goes after this class.

Agenda

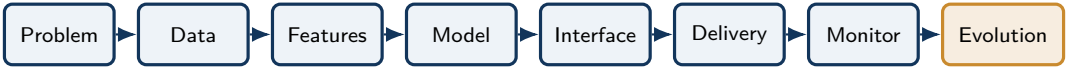
- Recap: what makes a requirement good
- Workshop structure and case assignment
- Elicitation: asking the right questions
- Writing functional and non-functional requirements
- Surfacing conflicts and trade-offs
- Peer review against a validation checklist

From theory to a written specification

Learning goals

- Apply elicitation techniques to a concrete case in one working session.
- Draft testable functional and non-functional requirements from a vague prompt.
- Identify a genuine conflict between requirements and propose an explicit trade-off.
- Use a validation checklist to critique a peer team's requirements.

Course Map and Running Cases



Today in the course

Focus: Turning requirements theory into a written, testable specification.
Bridge: This workshop applies the theory from the first half of Class 22 to a hands-on writing exercise.

Fraud case

Fraud track: write functional and non-functional requirements for the fraud scoring service, from ingestion to analyst review.

Pasteurization case

Pasteurization track: write functional and non-functional requirements for the abnormal-state alerting dashboard.

Course thread: Requirements written today should be traceable to the tests, deployment gates, and monitoring rules the course covers later.

Recap: qualities of a good requirement

- Necessary, testable, unambiguous, feasible, traceable.
- A requirement is not a feature wish; it is something a reviewer could check.

Weak versus better, one more time

"The system should be reliable" versus "the stream endpoint shall recover and resume within 5 seconds after a dropped connection."

Workshop structure

- Form small groups; each group picks one track: fraud scoring or pasteurization alerting.
- Time-box each stage; a rough spec beats an unfinished perfect one.
- Every stage below produces a short written artifact the group keeps.

1. Elicit (10 min)
2. Draft functional requirements (10 min)
3. Draft non-functional requirements (10 min)
4. Surface one conflict (5 min)
5. Peer review with the checklist (10 min)

Elicitation is structured discovery

- Elicitation means discovering requirements from users, operators, policies, data constraints, and business goals.
- Interviews, observation, document review, and prototype walkthroughs are all useful methods.
- The purpose is not only to collect wishes, but to reveal conflicts and hidden assumptions.

Good elicitation prompts for project teams

- Who acts on the prediction, and what action do they take?
- What happens if the model is unavailable for one hour?
- Which inputs may arrive late, missing, or malformed?
- When do labels appear, and who trusts them?
- Which requirement is actually a policy or compliance constraint?

Exercise 1: Elicitation notes

- Answer the prompts on the previous slide for your chosen track.
- Write raw notes, not polished sentences; contradictions between teammates are useful signal, not noise.
- Flag anything that sounds like a policy or domain constraint rather than a feature request.

From notes to functional requirements

Raw note

"The analyst needs to know why a transaction was flagged."

Functional requirement

The API shall return, alongside the score, the top 3 contributing features for each flagged transaction.

- A functional requirement names an observable system behavior, not a feeling.

Exercise 2: Write three functional requirements

- Using your elicitation notes, write three functional requirements for your track.
- Each one should name an input, an action, and an observable output.
- Avoid the words “should” and “easy”; prefer “shall” plus a concrete behavior.

Non-functional requirements deserve equal weight

Category	Question to answer with a number
Latency	How fast must a response or alert be, and for what fraction of cases?
Availability	How much downtime is tolerable, and what happens during it?
Auditability	What must be logged, and for how long?
Privacy	What data can be stored, shown, or shared, and with whom?

Exercise 3: Write two non-functional requirements

- Pick two categories from the table and write one requirement each.
- Every requirement must include a number: a time, a percentage, a retention period, or a rate.
- If you cannot put a number on it yet, write down what you would need to know to add one.

Conflicts between requirements are normal

- Fast response may conflict with more detailed explanations.
- High recall may conflict with low false positive rates.
- Privacy constraints may limit what data can be stored for monitoring or audit.
- Requirements engineering helps teams make these trade-offs explicit.

Exercise 4: Name one conflict

- Look at your functional and non-functional requirements together.
- Find one real tension between them, for example explanation detail versus latency.
- Write one sentence stating which side you would favor by default, and why.

Validation techniques that fit student projects

- Requirement reviews with a checklist for ambiguity and measurability.
- Use-case walkthroughs for edge cases and operator behavior.
- Prototype validation with a minimal dashboard or API client.
- Acceptance test drafting before full implementation.

Exercise 5: Peer review

- Swap your written requirements with another group.
- For each requirement, check: necessary, testable, unambiguous, feasible, traceable.
- Mark anything that fails a check and suggest a one-line fix.

Ground your requirement in something real

Repo activity

- Fraud track: anchor requirements to the fields the API already accepts and returns, as seen in ‘Class6_model_api.py‘.
- Pasteurization track: anchor requirements to the fields already streamed by ‘pasteurization/serving.py‘, such as state, T, pH, and dTdt.
- A requirement that cannot be tied to an existing field or endpoint is either a new feature request or still too vague.

Discussion prompt

Question

Which of your non-functional requirements would be hardest to test automatically, and why?

- This surfaces the gap between writing a requirement and actually verifying it.
- It is a preview of the testing and monitoring gates later in the course.

What a strong requirements artifact looks like

- A short list of functional requirements naming input, action, and output.
- A short list of non-functional requirements, each with a number attached.
- At least one documented conflict and the team's default resolution.
- Peer-review marks showing the requirements survived a second reading.

Papers and materials

Papers and materials

Sommerville, Software Engineering; IEEE recommended practice for software requirements specifications; User stories and acceptance criteria guide; Practical overview of functional and non-functional requirements.

From This Class to the Next

What stays with us

- A requirement earns its place only if someone could check whether it was met.
- Elicitation, drafting, and peer review are fast enough to run in a single session.
- Conflicts between requirements are normal and should be written down, not hidden.

Project use

Keep today's written requirements; they are the seed of the project proposal work in Class 33 and Class 35.

Next connection

Class 23 moves from what the system must do to how the team organizes work to build it, starting with process models and agile foundations.

Course thread: Today's artifact is the first concrete deliverable in the course thread that eventually becomes each team's own project.